



Universidade de Brasília (UnB)
Departamento de Ciência da Computação (CIC)
CIC0124 - Redes de Computadores - Turma 03
Professor: Roberto Rodrigues Filho

Alunos:
Eduardo Agostinho Freitas Fernandes - 211020849
Fernando Vera Cruz de Oliveira Ferreira Araujo - 221018915
Gustavo Freitas de Sousa - 202037669
Vinicius Chaves - 211060764

Relatório de implementação de uma versão simplificada de um serviço similar ao Onedrive

1- Descrição geral da solução e fluxo da simulação

Foi implementado um sistema distribuído Peer-to-Peer (P2P) para sincronização de arquivos em rede. O projeto se iniciou como um sistema que operava de forma descentralizada pura, utilizando broadcast UDP para descoberta de outros Nós e troca de índices completos por meio de TCP a cada 10 segundos, mas essa abordagem gerava uma sobrecarga de rede e I/O, além de em alguns testes ter duplicado arquivos (quando ele era renomeado enquanto era baixado por outro Nó).

Por causa disso, o projeto sofreu uma reestruturação completa, onde foi implementado uma arquitetura híbrida baseada em eventos. O fluxo passa a ser coordenado por um servidor central (*tracker.py*) que apenas repassa os comando (*CREATE, MODIFY, DELETE, RENAME*), enquanto a transferência dos arquivos vai ocorrer de forma direta P2P entre os Nós (*main.py*) por meio de uma conexão TCP (foi decidido manter a transferência de arquivos dessa forma para que o servidor não viesse a ser motivo de gargalo). Toda a estrutura foi isolada e simulada utilizando containers Docker em uma rede Bridge, o que garantiu a simulação de múltiplos endereços IP distintos operando em concorrência sem muito trabalho.

2- Arquitetura e variáveis principais

O servidor (Tracker) fica escutando na porta TCP 9000, e ele que mantém o dicionário (*nos_conectados*) para gerenciar as threads ativas de cada nó registrado e utiliza uma estrutura de lock (*threading.Lock()*) para evitar colisões durante a manipulação das conexões.

Referente aos Nós, é definido um valor único para cada instância (*uuid.uuid4*), a pasta de persistência (*PASTA_COMPARTILHADA*) e uma conexão global (*socket_tracker_global*) como um cliente TCP de longa duração. Para a comunicação, está sendo utilizado o DNS interno do Docker, e o endereço de IP sendo injetado via variável de ambiente (*TRACKER_HOST*)

3- Transição de estado e monitoramento da pasta

A detecção de mudança nos arquivos foi o que mais deu trabalho e mais precisou ser modificado para ter uma estabilidade mínima.

3.1- Otimização e escolha de hash MD5

Para identificar as mudanças nos arquivos, era necessário algo único para que fosse feita a comparação, e a escolha do MD5 foi basicamente por questão de performance, já que não estamos prezando pela segurança (nesse primeiro momento), e ele oferece um custo computacional bem menor. Além disso, a função de hash foi feita utilizando leitura fragmentada em blocos (de 4096 bytes) para impedir que o SO tente carregar arquivos grandes inteiros na RAM.

3.2 - Cache de metadados e redução de gargalos

Inicialmente, quando era feito o escaneamento da pasta, era calculado o MD5 de todos os arquivos do diretório em loop, e quando foram feitos testes com arquivos grandes, essa abordagem fez que ocorresse um “DDoS” no próprio disco rígido. Para resolver esse problema, foi adotado uma verificação de cache de metadados, a função para de gerar a hash de todos os arquivos da pasta, e passa a verificar primeiro a data de modificação (*os.path.getmtime*) e o tamanho (*os.path.getsize*) de cada arquivo, fazendo que a leitura binária e a geração do hash só ocorre se esse metadados forem diferentes dos apresentados no estado anterior, reduzindo o tempo de varredura de vários segundos para menos de um segundo (com um arquivo de 1.1GB na pasta, cada verificação estava demorando cerca de 1 minuto antes de implementar a verificação do cache)

3.3 - Detecção de renomeação

Para evitar transferências desnecessárias, foi implementada uma lógica de cruzamento dos hashes, isolando os arquivos que sumiram e que apareceram, comparando os hashes das duas listas, e caso um arquivo ausente tenha a mesma assinatura de um arquivo novo, não é gerada uma ação de exclusão e uma de download, apenas uma ação de (*RENAME*).

4 - Protocolo de comunicação

A substituição da requisição dos dicionários inteiros, por apenas eventos, precisou de um controle do tráfego rigoroso, e a escolha do protocolo TCP para a transferência dos arquivos é o ideal, já que o TCP garante a entrega dos pacotes (se fosse implementado com UDP os arquivos poderiam ser corrompidos durante a transferência).

4.1 - Resolução do TCP Sticky Packets

Devido ao TCP se basear no fluxo de bytes, a junção do envio de informações de controle por meio de JSON e os bytes puros do arquivo resultaram na mistura dos pacotes (sticky packets), o que causava um erro no parser de quem recebia. Inicialmente foi utilizado um *time.sleep* para forçar a segmentação, mas ao final, foi implementado os cabeçalhos de tamanho fixo. Para isso, foi utilizada a biblioteca *struct* para empacotar o tamanho exato da mensagem JSON em 4 bytes formatados, e agora o receptor faz a leitura dos primeiros 4 bytes, separando os metadados, do tráfego binário que vem depois.

5 - Tolerância a falhas

Implementamos alguns mecanismos de tolerância a falhas para ter uma maior coerência dos dados replicados entre os Nós.

5.1 - Extensão temporária e condição de corrida

Durante um dos testes (quando trabalhava com arquivos grandes), foi observado que acontecia uma condição de corrida: enquanto o Nó estava fazendo o download do arquivo, a rotina de monitoramento lia esse arquivo incompleto, gerava um novo hash e emitia um outro evento de novo arquivo, fazendo com que tivesse eco na rede, causando o fechamento das conexões (Errno 32: Broken Pipe).

Para resolver essa condição, foi adicionado uma extensão temporária (*.tmp*) que fica no arquivo enquanto ele é baixado, e o monitor ignora os arquivos com essa extensão, e quando o download termina, e a contagem dos bytes recebidos é igual ao tamanho definido no cabeçalho, a extensão é removida.

5.2 - Idempotência operacional

Cada ação recebida via Tracker passa por uma checagem prévia do estado. Em operações de download (CREATE/MODIFY), o nó primeiramente verifica localmente se o arquivo já existe e se possui a mesma paridade de tamanho. Essa abordagem faz com que o protocolo assegure que o processamento repetido da mesma mensagem não gere tráfego duplicado, prevenindo loops infinitos.

6 - Dificuldades encontradas

As principais dificuldades que foram encontrados durante o desenvolvimento do projeto, foram desde o esgotamento de memória (que foi resolvido dividindo o arquivo em blocos), pacotes grudados (que foi resolvido com o uso de cabeçalho), loops de download infinito (resolvido com a extensão .tmp), decisão da escolha entre o TCP ou o UDP (inicialmente o projeto se basearia em broadcast UDP, mas devido a não “segurança” das transferências, foi escolhido o TCP), utilização de um servidor central (inicialmente o projeto não teria um servidor central, e os Nós trocariam o índice completo toda vez, mas isso gerou uma sobrecarga de tráfego, além de duplicações dos arquivos)

7 - Conclusão

A implementação do sistema P2P híbrido cumpre com a proposta de entregar uma versão simplificada de um serviço similar ao OneDrive. A transição de uma arquitetura descentralizada pura para um modelo orientado a eventos se mostrou essencial para eliminar a sobrecarga da rede e otimizar o uso dos discos rígidos. As soluções técnicas aplicadas ao longo do desenvolvimento, como o cache de metadados, a leitura fragmentada em blocos e a prevenção de condições de corrida através do uso da extensão temporária, garantiram a estabilidade do projeto.

8 - Instruções de execução

O ecossistema do projeto foi encapsulado utilizando o Docker para garantir o isolamento da rede P2P e facilitar a avaliação. Não é necessária a instalação de bibliotecas Python locais.

Pre-requisitos:

Docker instalado e funcionando

Inicialização:

Abra o terminal na pasta raiz do projeto (a pasta que contem os arquivos: main.py, tracker.py, Dockerfile, docker-compose.yml)

Execute o comando **docker-compose up --build**

Ao executar o comando, vai ser exibido os logs de criação do ambiente docker, o tracker vai ser iniciado, e quando o mesmo terminar, vão ser criadas 3 containers na raiz do projeto para representar os Nós (no_a, no_b, no_c).

A partir desse momento, o sistema já está funcionando, quando um arquivo é adicionado, modificado, renomeado ou deletado em uma das pastas, a mesma emite um evento para o tracker, que vai notificar os outros nós para que eles façam as ações necessárias no arquivo, e todas essas operações são descritas no log que aparece no terminal.